INT161 · Basic Backend Development

RESTful API

The REST constraints, naming resources with URIs, and splitting code into layers without a framework.

What is REST | Principles and Constraints | Uniform Interface | URI Naming

Layered System | Layers in Pure Node.js | Glossary

01

What is REST

This is what the whole course builds: a **REST API** that talks to the **front end** in **JSON**. Tools change, the shape stays.

- REST stands for REpresentational State Transfer. It was defined by Roy Fielding, who also helped shape **HTTP**.
- REST API and **RESTful** API are used together. The lecturer's reading: a RESTful API follows the rules strictly, and the name hints at "relaxed" after the heavy **SOAP** days.
- API means Application Programming Interface: the part of a program that other programs call. Ours is called by the front end.
- A REST **server** gives access to **resources**. For us a resource is a database table (an **entity**). It could also be a file, or even an air conditioner controlled by an IoT device.
- What the front end really does with a resource is **CRUD**: create is INSERT, read is SELECT, update is UPDATE, delete is DELETE.
- Each resource is named by a **URI**. The **representation** sent between the two sides can be text, XML or JSON. JSON is what everyone uses.

> **Boilerplate**
>
> Writing CRUD for ten tables gives ten copies of the same code, with only the table and fields changed. That repeated code is called boilerplate. **Frameworks** solve it. This class has no framework yet, so you feel the pain first.

Where does the front end live? It is deployed on its own server. The first **request** goes there and the whole front end project downloads into the **browser**. After that the browser calls the **back end**. (The browser then checks cross-origin calls. That is CORS, taught in another course. When you test with **Postman** there is no front end, so no CORS.)

02

Principles and Constraints

`Client-server`

Two separate programs. The back end serves resources, the front end asks for them.

`Stateless`

The server does not remember **clients**. Each request carries what it needs.

`Uniform interface`

Everyone calls the API the same way. This is the one we study.

`Layered system`

Code is split into layers by job. The other one we study.

`Cacheable`

A **response** says whether it may be reused. We skip it in this course.

`Code on demand`

Optional. The server may send code for the client to run. Rarely done.

- **Stateless** in detail. Application state is what the app remembers about each user: who is logged in, what is in the shopping cart. In the old multi-page style, the server kept it in a session. In REST, the back end keeps none. The front end holds it and sends what is needed with each request.

- That makes the front end harder to write, but the back end stays simple and easy to scale.
- **Cacheable**, by the fridge analogy: the market is storage, the fridge is memory. Keep what you cook often in the fridge. A hit means it was there. A wrong guess wastes space, so the guess must be good. Libraries do this for you.

03

Uniform Interface

Uniform means a uniform: Monday is the school uniform, Thursday is the scout uniform, and you never wonder what to wear. The same here. A front end developer and a back end developer need no meeting to agree how to call an API. The standard already decides.

1. Identify the resource

By a URI that is a noun, not an action.

2. Exchange representations

Standard formats. JSON, with Content-Type application/json.

3. Self-descriptive messages

The HTTP method says what to do. **Headers** describe the data.

4. HATEOAS

Responses carry links to the next actions. Optional, rarely followed.

The URI names the resource. The method chooses the action. One name, four actions. Try the pairs.

Method GET POST PUT PATCH DELETE

Target Collection One item

`GET /api/v1/products`

Same as SQL

`SELECT * FROM products`

JSON body

None

Success

`200 OK`

List all products. Returns a JSON array. Big tables should be paged.

- Before this standard, a front end called /create-user. Now it calls the resource and picks the action with the method.
- Many teams delete by marking a row as deleted instead (a **soft delete**), so the data can still be traced.
- A request that creates or updates needs a Content-Type of application/json.

HATEOAS: data plus links

```
{
  "id": 1,
  "name": "Somchai",
  "links": [
    { "rel": "orders", "href": "/api/v1/customers/1/orders" },
    { "rel": "delete", "href": "/api/v1/customers/1" }
  ]
}
```

04

URI Naming

Your **endpoint** is the URI plus the method. These rules are why marks go missing: the lecturer warned that /product without the s can cost points in the exam.

Pick a URI to see the verdict.

`/api/v1/products`

```
/api/v1/getProducts
```

```
/api/v1/Products
```

```
/api/v1/product
```

```
/api/v1/order_items
```

```
/api/v1/products/
```

```
/api/v1/devices.xml
```

```
/api/v1/customers/1/orders
```

- Nouns, never verbs. A table is a noun, so a resource is too.
- Plural everywhere, even for one item. /products, /products/1.
- Lowercase, with hyphens between words (**kebab-case**). Underscores are the database habit.
- Slash goes deeper: /customers/1/orders is the orders of customer 1. No trailing slash, no file extension, no CRUD words in the path.
- Start the path with the app and a version, such as /api/v1. If you later change what an endpoint sends, publish /api/v2 and let each front end switch when it is ready. Old callers keep working.
- A **collection** returns a JSON array. One item returns one JSON object.

> ⚠ **Real life**
>
> You may skip the version if you control both sides and change them together. Even then, a front end already in use cannot be forced to update at once, so a version is the safe habit.

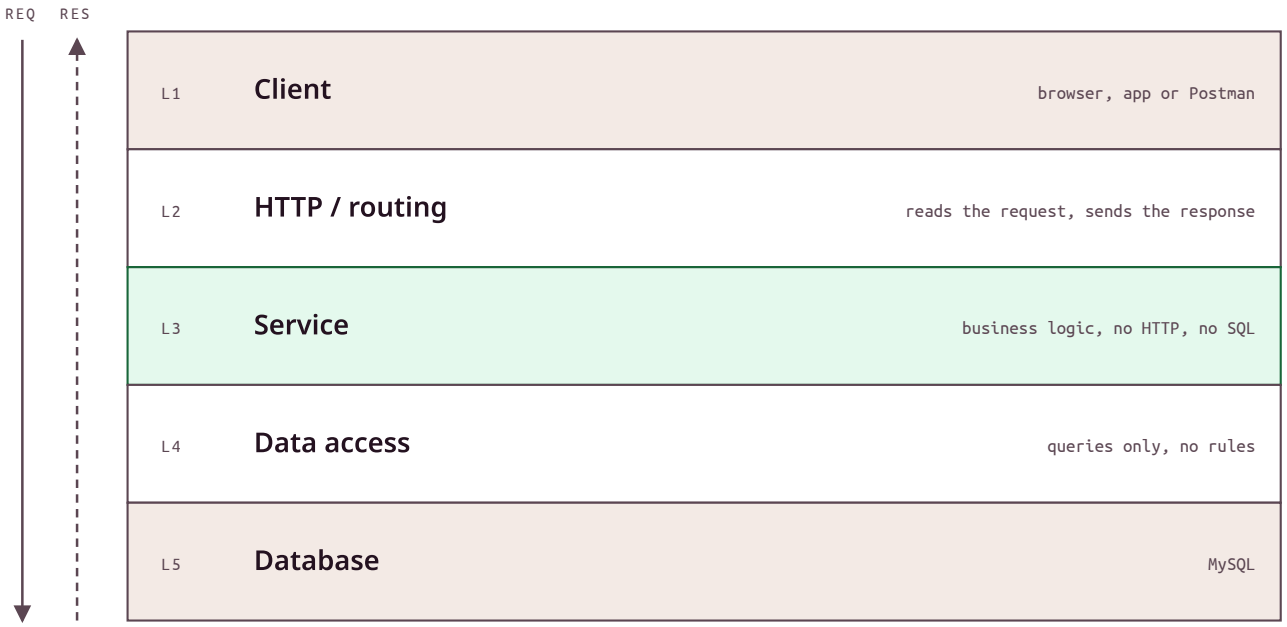
Using AI

The lecturer asked a browser AI to design the customer, product and order endpoints. It followed the same rules: nouns, lowercase, /api/v1, one method per action, and 200, 201, 400, 401 as **status codes**. Use AI to explain a word you do not know, not to write the code you are supposed to learn. You must know enough to spot what it got wrong.

05

Layered System

A **layered system** splits the program by job. This is separation of concerns: the person writing one layer does not need to care about the others.



`HTTP / routing layer`

Talks to the front end. Reads the method and **URL**, turns the JSON **body** into a JavaScript object, sends the response.

Service layer

Business logic. For example: may this student enrol in a 6th subject with this grade? If not, tell them to file a request first.

Data access layer

Only insert, update, delete and find. No rules, no HTTP.

- The goal is scalability and reuse. Swap a framework in for one layer without touching the others.
- JSON and a JavaScript object look alike, but they differ. JSON field names need double quotes. JavaScript field names do not. The top layer converts one into the other.

06

Layers in Pure Node.js

No framework yet. The http **module** does the HTTP part, and you lay out the layers yourself.

project layout

```
src/
├── server.js        native HTTP server setup
├── router.js        route matching and HTTP handling
├── services/        business logic, no framework
└── repositories/    data access (file system or DB)
```

The lecturer built the bottom layer first, with an array standing in for the database. The four functions of a **repository** are CRUD again.

repositories/subject-repository.js

```
const subjects = [];
export function findById(id) {
  return subjects.find((s) => s.id === id);
}
export function create(subject) {
  if (subjects.findIndex((s) => s.id === subject.id) >= 0) return false; // duplicate id
  subjects.push(subject);
  return subject;
}
export function update(id, subject) {
  const idx = subjects.findIndex((s) => s.id === id);
  if (idx < 0) return false; // fail fast
  subjects[idx] = { ...subject, id };
  return subjects[idx];
}
export function remove(id) {
  const idx = subjects.findIndex((s) => s.id === id);
  if (idx < 0) return false;
  subjects.splice(idx, 1);
  return true;
}
```

- There is no auto-generated key yet, so create checks for a duplicate id first and returns false if it exists.
- update and remove use findIndex. A negative index means not found. They return false, or the item, or true. The caller can tell whether it worked.
- Check the failure first and return early. That style is called fail fast.

> **Test layer by layer**
>
> Test the repository on its own before you write the next layer. Call each function from a test file and print the result. When something breaks later, you already know the lower layer works, so the bug is in the new code.

- At the top, the **router** maps each method and URL to a **service** call. Test with Postman: GET /subjects, GET /subjects/INT102, DELETE answers 204 No Content with no body. POST and PUT stay "under construction" until the body is read (next time).

07

Glossary

API
: Application Programming Interface: the doors a program offers to other programs.

back end
: The part on the server: it handles data, rules and the database, and answers the front end with JSON.

body
: The data a request carries after the headers. For an API it is usually JSON.

Boilerplate
: Code that is almost the same every time, with only the names changed. Frameworks exist to remove it.

browser
: The client most people use, such as Chrome, Edge or Safari.

Business logic
: The rules of the application itself: checks, calculations, and what may happen when.

Cacheable
: Keeping data you use often in fast memory so you skip the slow database. A hit means it was already there.

clients
: The side that asks. Usually a browser, but it can be any program or device.

collection
: A resource name that returns many items, like /products. The answer is a JSON array.

CRUD
: Create, Read, Update, Delete: the four basic things you do with data.

endpoint
: One method plus path an API answers, such as GET /api/subjects.

entity
: A database record as stored: a row of a table, with every column.

Frameworks
: A toolkit that makes a job easier but sets the frame you must write inside.

front end

The part users see and touch. It runs in the browser or on the phone.

Headers

Extra information about a message, sent as name: value lines, such as Content-Type.

HTTP

HyperText Transfer Protocol: the plain-text language a browser and a web server use to talk. One side sends a request, the other sends a response.

JSON

JavaScript Object Notation: a plain text format for data that both humans and programs can read.

kebab-case

Words joined by hyphens, like order-items. The convention for URIs.

layered system

Split the code into layers by job, so each layer only talks to its neighbour.

module

A file of code you load into another file with import.

Postman

A tool for sending any HTTP method to your API, so you can test it before the front end exists.

repository

The layer that reads and writes the database. No business rules, no HTTP.

representation

The format a resource is sent in, such as text, XML or JSON. JSON is the usual choice.

request

What the client sends: method, URL, headers and sometimes a body.

resources

A thing the API gives access to, such as a product or a customer. In this course it maps to a database table.

response

What the server sends back: status code, headers and a body.

REST

A style for web APIs: resources named by URLs, actions chosen by HTTP method.

RESTful

An API that follows all the REST rules, not just uses HTTP and JSON.

router

Matching an HTTP method and a URL path to the function that handles it.

server
: A program (or the machine running it) that waits for requests and answers them.

service
: The layer that holds the business rules. It calls repositories and knows nothing about HTTP.

SOAP
: Simple Object Access Protocol: an older web service style that sends XML. Heavier, but supports digital signatures.

soft delete
: Mark a row as deleted instead of removing it, so the data can still be traced.

Stateless
: The server keeps nothing between requests. Each request must carry everything it needs.

status codes
: The number that tells the client how it went: 200 OK, 201 Created, 404 Not Found, 500 Server Error.

URI
: Uniform Resource Identifier: the text that names a resource, like /products/1. A URL is a URI that also says where to find it.

URL
: The address of a resource: protocol, host, port and path, such as http://localhost:3000/user.